

Lab 5: Stacks

Algorithms and Data Structures (010153523) | Semester 1/2026

Duration: 2 hours (in-lab) | **Topic:** Stack ADT, array-based implementation, applications

Objectives

- Define the Stack ADT and its `push`, `pop`, `peek`, `size`, `isEmpty` operations.
- Implement a stack backed by a fixed-capacity array.
- Handle stack overflow and underflow with appropriate exceptions.
- Apply stacks to: bracket matching, expression evaluation, and reversing a sequence.

Quick Reference

Stack interface.

```
public interface Stack<E> {
    int size();
    boolean isEmpty();
    void push(E element) throws IllegalStateException;
    E pop() throws java.util.EmptyStackException;
    E peek() throws java.util.EmptyStackException;
}
```

Array-based push/pop.

```
// push: data[top++] = e;    (check capacity first)
// pop:  return (E) data[--top];
// peek: return (E) data[top-1];
```

LIFO rule. Last element pushed is the first one popped.

Complexity. All Stack operations: $O(1)$.

Quick Experiments (Try It)

Try It 1: Overflow and underflow

Create an `ArrayStack<Integer>` of capacity 3. Push four elements. What happens? Now pop from an empty stack. What exception is thrown? Write a try-catch block that handles both gracefully.

Try It 2: The JVM method stack

Write a method that calls itself unconditionally (no base case). Run it and observe the exception. How does this demonstrate that the JVM uses a stack for method calls?

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Array-Based Stack (35 min)

Implement `ArrayStack<E>` backed by an `Object[]` of configurable capacity.

- `push` throws `IllegalStateException("Stack is full")` when at capacity.
- `pop` and `peek` throw `EmptyStackException` when empty.
- `toString()` prints bottom to top separated by `|`, e.g. `[1|2|3]` where 3 is on top.

Test with 6 pushes and 4 pops. Verify `size()` after each operation.

In-Lab Exercise 2: Bracket Matching (30 min)

Write `boolean isBalanced(String expr)` that returns `true` if every opening bracket is matched by the correct closing bracket. Support three pairs: `()`, `[]`, `{}`.

- Push every opening bracket; on a closing bracket, pop and check for a match.
- Return `false` on mismatch or if the stack is non-empty at the end.

Test with: `"([{}])"`, `"{[()]}"`, `"((("`, `""`.

In-Lab Exercise 3: Reverse a String (15 min)

Using your stack, write `String reverse(String s)` that pushes each character then pops them all into a `StringBuilder`. Verify `reverse("hello") = "olleh"`.

In-Lab Exercise 4: Evaluating Postfix (RPN) Expressions (40 min)

Write `int evalPostfix(String expr)` where `expr` is space-separated with single-digit integers and operators `+`, `-`, `*`, `/`. Push digits; on an operator, pop two operands, apply, push result. Test: `"3 4 +"` (`= 7`), `"5 1 2 + 4 * + 3 -"` (`= 14`).

AI Collaboration

Suggested prompts:

- “Trace the bracket-matching algorithm step by step on `'{[()]}'`.”
- “Explain why a stack is the natural data structure for evaluating postfix expressions.”

Verify: Ask the AI for a postfix expression that evaluates to 42. Trace it manually through your algorithm before running the code.

Take-Home (Paper Work). Solve the following on paper without using a computer or IDE. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to trace and reason about code during the written exam.

Trace & Predict 1: Stack trace

Trace the stack contents (bottom to top) after each operation:

```
push(5); push(3); push(8); pop(); push(2); peek(); pop(); pop();
```

What is returned by each `pop()` call?

Trace & Predict 2: Bracket matching trace

Trace `isBalanced("([{}])")` step by step. For each character show: the character, the action (push/pop/mismatch), and the stack contents. Does the method return `true` or `false`?

Find the Bug 1: Broken pop

Find and fix all errors (assume `top` starts at 0 and `push` does `data[top++] = e`):

```
public E pop() {  
    if (isEmpty()) throw new java.util.EmptyStackException();  
    E val = (E) data[top];  
    top--;  
    return val;  
}
```

Write Code on Paper 1: Infix to postfix

Write pseudocode (or Java) for converting a simple infix expression (single-digit operands, operators `+` and `*` only, no parentheses) to postfix using a stack. State the rule for when to pop vs. push an operator.

Submission. Save files as `lab5_ex*.java` and submit on the course site.